

PES UNIVERSITY, BANGALORE

Department of Computer Science and Engineering

Jackfruit Phase-1

Software Requirements Specification

for

Pocket Packet Sniffer

Version 1.0 approved

Prepared by: Ankita S, Anusha Gupta, Charan M, Vaghasiya Akshar Arvindbhai

Group Number: 4

PES University, Bangalore

Date Created: 07 September 2026

Table of Contents

Table of Contents.....	2
Revision History.....	3
1. Introduction	4
1.1 Purpose	4
1.2 Intended Audience and Reading Suggestions.....	4
1.3 Product Scope	4
1.4 References	4
2. Overall Description.....	4
2.1 Product Perspective	4
2.2 Product Functions	4
2.3 User Classes and Characteristics	5
2.4 Operating Environment	5
2.5 Design and Implementation Constraints	5
2.6 Assumptions and Dependencies	5
3. External Interface Requirements	5
3.1 User Interfaces.....	6
3.2 Software Interfaces	6
3.3 Communications Interfaces	6
4. Analysis Models	7
5. System Features	7
5.1 Live Capture & Session Control.....	7
5.2 Real-Time Traffic Summary Dashboard	7
5.3 Anomaly & Suspicious-Activity Alerting	8
5.4 Session History, Storage & Export	8
6. Other Nonfunctional Requirements	10
6.1 Performance Requirements.....	10
6.2 Safety Requirements.....	10
6.3 Security Requirements.....	10
6.4 Software Quality Attributes	10
6.5 Business Rules.....	10
7. Other Requirements	10
Appendix A: Glossary	10
Appendix B: Field Layouts.....	12
Appendix C: Requirement Traceability Matrix	13

Revision History

Name	Date	Reason For Changes	Version
Ankita S, Anusha Gupta, Charan M, Vaghasiya Akshar Arvindbhai	07-Sep-2026	Initial SRS drafted from approved project synopsis	1.0

1. Introduction

1.1 Purpose

This document specifies the software requirements for the Pocket Packet Sniffer, a compact, portable network traffic monitoring device developed as a Software Engineering mini-project (Jackfruit Phase-1) at PES University, Bangalore, by Group 4. This is Version 1.0 of the SRS and covers the complete scope of the mini-project as approved in the project synopsis. This SRS describes the requirements for the entire system, not a subsystem.

1.2 Intended Audience and Reading Suggestions

This document is intended for the development team members (Ankita S, Anusha Gupta, Charan M, Vaghasiya Akshar Arvindbhai), the course instructor and teaching assistants evaluating the project, and any future contributor who needs to understand the system's functional and non-functional requirements. Section 2 gives an overall description of the product. Section 3 defines external interface requirements. Section 5 details each system feature with its functional requirements. Section 6 covers non-functional requirements. Readers primarily interested in scope and features should read Sections 1, 2, and 5; readers interested in quality attributes should focus on Section 6.

1.3 Product Scope

The Pocket Packet Sniffer is a compact, battery-friendly device built around a Raspberry Pi (with an ESP32 as an optional auxiliary hardware component), designed to capture live network traffic from a chosen interface, analyse it using the open-source Zeek network security monitoring engine, and present a simplified, human-readable summary through a lightweight Python-based interface. The product is intended to support networking and cybersecurity laboratory sessions, student learning, and small-scale network troubleshooting, allowing a student or junior network technician to get an immediate overview of network activity on a segment without setting up full capture software on a laptop. The system integrates Zeek for protocol parsing and detection rather than reimplementing it; the team's original contribution is the capture pipeline, portable hardware integration, summarisation and alerting logic on top of Zeek, and session management.

1.4 References

- Pocket Packet Sniffer — Synopsis / Project Proposal, Group 4, PES University, Bangalore.
- IMP Details on SE Mini-Project Guidelines: Evaluation, Synopsis and SRS 2026, Department of Computer Science and Engineering, PES University, Bangalore (Subject Code: UE24CS341A).
- Zeek Network Security Monitor documentation — <https://zeek.org>
- IEEE SRS Template (Jackfruit Phase-1 format), PES University.

2. Overall Description

2.1 Product Perspective

The Pocket Packet Sniffer is a new, self-contained product; it is not a follow-on to an existing system. It integrates an established open-source analysis engine (Zeek) rather than reimplementing protocol parsing, which is standard practice for a project of this scope. The system consists of four logically separate layers: (1) a capture layer running on Raspberry Pi hardware (with optional ESP32 auxiliary hardware) that acquires raw packets from a network interface; (2) a Zeek analysis layer that parses protocols, generates structured logs, and applies detection scripts; (3) a Python-based control/UI layer that reads Zeek's output and renders a live summary and alerts; and (4) a storage/export layer that persists and exports captured sessions. The device operates standalone at the point of network interest and does not depend on connectivity to any external server.

2.2 Product Functions

At a high level, the product allows a user to:

- Select a network interface and start, stop, or time-bound a live packet capture session, optionally applying a basic filter.

- View a real-time, simplified summary of network activity (protocol breakdown, top talkers, connection counts) while a capture is running or after it ends.
- Enable alerting to have the system flag predefined suspicious traffic patterns (e.g., port-scan behaviour) with timestamp, source, and reason.
- Browse, retrieve, and export past capture sessions (raw .pcap and/or Zeek logs) for offline analysis in tools such as Wireshark.

2.3 User Classes and Characteristics

The primary user class is networking and cybersecurity students and teaching assistants running lab sessions — users with basic networking knowledge but not necessarily prior Wireshark or Zeek expertise. A secondary user class is junior IT or network administrators who need a quick, low-effort way to check traffic on a segment. Both classes are expected to use all functional features of the system; there is no distinct administrator/end-user privilege split, since the device is single-user and operated locally at the point of capture.

2.4 Operating Environment

The system runs on a Raspberry Pi (or equivalent single-board computer) running a Linux-based operating system, with Zeek installed as the packet analysis engine and Python 3 powering the control application and user interface. An ESP32 microcontroller may be used as an auxiliary hardware component where the final hardware design requires additional network interfacing. The device is battery-powered for portability and connects to the target network segment via a wired or wireless network interface. No dependency on cloud services or external servers is required for core operation; local storage (e.g., microSD card) is used to persist capture sessions.

2.5 Design and Implementation Constraints

- Hardware constraint: must run on Raspberry Pi-class hardware with limited CPU, memory, and storage compared to a full laptop/workstation.
- Technology constraint: packet analysis must be performed via Zeek rather than a custom-built protocol parser.
- Language/tooling constraint: the control application and UI must be implemented in Python.
- Data format constraint: raw captures must be stored in standard .pcap format so they remain compatible with external tools such as Wireshark.
- Development process constraint: the project must follow Agile methodology with sprint-based delivery, and use Git/GitHub for version control with active use of a backlog/issue tracker.
- Power/portability constraint: the device must be battery-friendly and self-contained, without requiring a full laptop setup at the point of capture.
- Security constraint: captured traffic data must be stored locally by default with no automatic transmission off the device.

2.6 Assumptions and Dependencies

- It is assumed that Zeek is available and correctly installable on the chosen Raspberry Pi hardware and Linux distribution.
- It is assumed that the network interface used for capture supports promiscuous/monitor mode as required for packet capture on the target segment.
- The project depends on Zeek's built-in and extended detection scripts for identifying suspicious traffic patterns (e.g., port-scan detection); custom signature development beyond what Zeek provides is out of scope.
- It is assumed that capture activity is performed with appropriate authorisation on networks the team is permitted to monitor (e.g., lab environments).
- The project assumes availability of sample and lab-generated traffic for testing, since some detection scenarios (e.g., port scans) cannot be reliably tested against production networks.

3. External Interface Requirements

3.1 User Interfaces

The system exposes a lightweight Python-based interface (terminal-based or simple GUI/web dashboard, depending on final design) providing: a capture control screen to select an interface and start/stop/filter a session; a live-updating dashboard showing protocol breakdown, top talkers, and connection counts; an alerts view listing flagged suspicious events with timestamp, source, and reason; and a session browser for viewing, retrieving, and exporting past sessions. The interface is designed to be understandable to a user without prior Zeek or Wireshark expertise, favouring simplified, readable summaries over raw log output.

3.2 Software Interfaces

The system interfaces with Zeek (the network security monitoring engine), which produces structured log files such as `conn.log`, `dns.log`, and `http.log`. The Python control application reads these logs in real time to generate summaries and alerts. The capture layer interfaces with the underlying operating system's packet capture facilities (e.g., `libpcap`) to acquire raw traffic and write `.pcap` files. Exported sessions are formatted for compatibility with external analysis tools such as Wireshark. Local storage uses a file-based or lightweight database structure to index session metadata.

3.3 Communications Interfaces

The device captures traffic over the network interface connected to the target segment (wired Ethernet or Wi-Fi, depending on hardware configuration). No mandatory external network communication (e.g., cloud upload) is required for core functionality, consistent with the requirement that captured data be stored locally by default. Where a future remote-dashboard capability is implemented (see Future Scope), communication would occur over the local network only.

4. Analysis Models

The overall data flow follows a linear pipeline: Network Interface → Capture Layer (Raspberry Pi / optional ESP32) → Zeek Analysis Engine (protocol parsing, structured logs, detection scripts) → Python Control Application (aggregation, summary rendering, alert generation) → Local Storage (session index, .pcap and Zeek logs) → Export (to external tools such as Wireshark). Each of the four functional features described in Section 5 maps to one stage or cross-cutting concern of this pipeline, and each is owned by one team member as described in the project's product ownership plan.

5. System Features

5.1 Live Capture & Session Control

5.1.1 Description and Priority

Allows the user to select a network interface and start, stop, or time-bound a live packet capture session, optionally applying a basic protocol filter. Priority: High — this is the entry point of the entire pipeline and all other features depend on it. Owner: Ankita S.

5.1.2 Stimulus/Response Sequences

- User selects a network interface and optionally sets a duration or protocol filter → system validates the interface is available and ready.
- User starts a capture → system begins streaming packets into Zeek in real time and writes a raw .pcap file to local storage.
- User stops a capture (or the set duration elapses) → system cleanly terminates the capture and finalises the .pcap file and associated Zeek logs.

5.1.3 Functional Requirements

REQ-1.1: The system shall allow the user to select an available network interface from a list before starting a capture.

REQ-1.2: The system shall allow the user to start and stop a capture session on demand.

REQ-1.3: The system shall allow the user to optionally set a maximum capture duration, after which the session ends automatically.

REQ-1.4: The system shall allow the user to optionally apply a basic filter (e.g., by protocol) to the capture.

REQ-1.5: The system shall stream captured packets to Zeek in real time for the duration of the session.

REQ-1.6: The system shall write the raw packet capture to local storage in standard .pcap format.

REQ-1.7: The system shall handle interface disconnects during a capture gracefully, without crashing the device, and shall notify the user of the interruption.

5.2 Real-Time Traffic Summary Dashboard

5.2.1 Description and Priority

Continuously reads Zeek's structured logs and presents a simplified, live-updating summary of network activity through the Python interface. Priority: High — this is the primary way a user derives value from a capture without reading raw Zeek logs. Owner: Anusha Gupta.

5.2.2 Stimulus/Response Sequences

- A capture session is active or has just ended → system continuously reads Zeek's structured logs (conn.log, dns.log, http.log, etc.).
- New log entries are detected → system aggregates statistics (protocol breakdown, top talkers, connection counts) and updates the summary view.
- User views the dashboard at any point during or after a capture → system displays the current aggregated summary.

5.2.3 Functional Requirements

REQ-2.1: The system shall continuously read Zeek's structured log output while a capture session is active.

REQ-2.2: The system shall compute and display a protocol breakdown of observed traffic.

REQ-2.3: The system shall compute and display a list of top talkers (hosts generating the most traffic or connections).

REQ-2.4: The system shall compute and display connection counts aggregated over the session.

REQ-2.5: The system shall update the displayed summary in real time (or at short, regular intervals) while a capture is running.

REQ-2.6: The system shall present the summary in a simplified textual/graphical form understandable without prior Zeek or Wireshark expertise.

5.3 Anomaly & Suspicious-Activity Alerting

5.3.1 Description and Priority

Uses Zeek's detection capabilities, together with selected project-specific logic, to flag predefined suspicious traffic patterns (e.g., port-scan behaviour) and present them as a prioritised, readable alert list. Priority: High — this differentiates the product from a plain traffic monitor. Owner: Charan M.

5.3.2 Stimulus/Response Sequences

- User enables alerting for a session → system activates the relevant Zeek detection scripts (standard and/or extended).
- Zeek or the project-specific detection logic identifies a suspicious pattern → system generates an alert entry with a timestamp, source, and reason.
- User reviews the alert list → system displays alerts in a prioritised, human-readable format.

5.3.3 Functional Requirements

REQ-3.1: The system shall allow the user to enable or disable alerting for a capture session.

REQ-3.2: The system shall use Zeek's standard and/or extended detection scripts to identify predefined suspicious traffic patterns, including port-scan behaviour.

REQ-3.3: The system shall generate an alert entry containing a timestamp, source, and reason whenever a suspicious pattern is detected.

REQ-3.4: The system shall present alerts to the user in a prioritised, readable list rather than requiring manual inspection of raw connections.

REQ-3.5: The system shall restrict alerting to patterns already supported by Zeek's standard/extended scripts or defined project-specific detection logic; it shall not claim to detect arbitrary or novel attack types outside this scope.

5.4 Session History, Storage & Export

5.4.1 Description and Priority

Maintains a local index of past capture sessions and allows the user to browse, select, and export a session (as .pcap and/or Zeek logs) for offline analysis. Priority: Medium-High — required for the system to be useful beyond a single live session. Owner: Vaghasiya Akshar Arvindbhai.

5.4.2 Stimulus/Response Sequences

- A capture session ends → system adds the session and its associated files/metadata to the local session index.
- User browses past sessions → system displays the list of stored sessions with relevant metadata (e.g., date, duration, interface).
- User selects a session and requests export → system packages the .pcap and/or Zeek logs for that session for offline use.

5.4.3 Functional Requirements

REQ-4.1: The system shall maintain a local index of all completed capture sessions and their associated files.

REQ-4.2: The system shall allow the user to browse and select any past capture session from the index.

REQ-4.3: The system shall allow the user to export a selected session's raw .pcap file and/or Zeek logs for offline analysis.

REQ-4.4: The system shall retain session data between application restarts and device reboots without data loss.

REQ-4.5: Exported .pcap files shall remain compatible with external analysis tools such as Wireshark.

6. Other Nonfunctional Requirements

6.1 Performance Requirements

The capture and log-processing pipeline shall keep up with live traffic on a typical small LAN/lab network segment without noticeable lag in the summary dashboard. The dashboard shall refresh at short, regular intervals during an active capture so that displayed statistics remain representative of current activity.

6.2 Safety Requirements

A capture session shall not crash the device, and the system shall handle network interface disconnects gracefully rather than entering an unrecoverable state. As the device may be used in shared lab environments, the system shall avoid actions (such as active probing) that could disrupt the network segment being monitored, restricting itself to passive capture and analysis.

6.3 Security Requirements

Captured traffic may include sensitive data; it shall be stored locally by default with no automatic transmission off the device. A basic security review of the alerting logic and data handling shall be conducted by the team as a shared responsibility. Access to stored sessions is assumed to be controlled by physical/OS-level access to the device, since the system is single-user and does not implement its own authentication layer in this phase.

6.4 Software Quality Attributes

- Usability: the dashboard and alerts shall be understandable to a user without prior Zeek or Wireshark expertise.
- Reliability: a capture session shall not crash the device and shall handle interface disconnects gracefully.
- Maintainability: there shall be a clear separation between the capture, analysis, UI, and storage layers, so each can be modified independently.
- Testability: each module shall be testable independently using pre-recorded .pcap files as input, without requiring live network hardware for every test run.
- Portability: the system shall run on Raspberry Pi-class hardware and shall not depend on proprietary or non-portable components beyond Zeek and Python.

6.5 Business Rules

- The system is intended for use on network segments the user is authorised to monitor (e.g., lab environments); it is not intended for unauthorised network surveillance.
- Only one capture session may be active on a given interface at a time in this phase of the project (multiple simultaneous interfaces are listed under Future Scope).
- Alerting is limited to suspicious patterns supported by Zeek's standard/extended scripts or explicitly defined project-specific logic; the system shall not be represented as a comprehensive intrusion detection or SIEM replacement.

7. Other Requirements

The project shall be developed using Agile methodology with sprint-based delivery (Sprint 0 – Setup, Sprint 1 – Backlog, Sprint 2 – Design, Sprint 3 – Core Implementation, Sprint 4 – Integration, Sprint 5 – Testing, Sprint 6 – Validation & Refinement), with GitHub (and Jira or a similar tool) actively used for user stories and backlog management, and CI/CD, code review, and basic security validation practised throughout, per the course guidelines (UE24CS341A). No database, internationalisation, or legal/regulatory requirements beyond local data storage and authorised-network-use are identified for this phase.

Appendix A: Glossary

Term	Definition
------	------------

Zeek	An open-source network security monitoring engine used for protocol parsing, structured logging, and behavioural/anomaly detection.
.pcap	A standard file format for storing captured network packets, compatible with tools such as Wireshark.
conn.log / dns.log / http.log	Structured log files produced by Zeek summarising connection, DNS, and HTTP activity respectively.
Top talkers	The hosts on a network segment responsible for the largest volume of traffic or the highest number of connections.
Port-scan behaviour	A traffic pattern in which a single source probes multiple ports/hosts in quick succession, commonly associated with reconnaissance activity and flagged by Zeek's detection scripts.
Raspberry Pi	A single-board computer used as the primary hardware platform for the capture and analysis pipeline.
ESP32	A low-cost microcontroller with networking capability, used as an optional auxiliary hardware component where the final hardware design requires it.

Appendix B: Field Layouts

Sample field layout for a session record stored in the local session index:

Field	Length	Data Type	Description	Is Mandatory
Session ID	16	Alphanumeric	Unique identifier for the capture session	Y
Interface Name	20	String	Network interface used for the capture	Y
Start Timestamp	20	DateTime	Date and time the capture started	Y
End Timestamp	20	DateTime	Date and time the capture ended	N
Duration (sec)	10	Numeric	Total capture duration in seconds	N
Filter Applied	50	String	Basic protocol filter applied, if any	N
PCAP File Path	255	String	Local file path to the raw .pcap capture	Y
Zeek Log Path	255	String	Local file path to the associated Zeek logs	Y
Alert Count	6	Numeric	Number of alerts generated during the session	N
Export Status	15	String	Whether the session has been exported	N

Appendix C: Requirement Traceability Matrix

Sl. No	Requirement ID	Brief Description	Architecture Reference	Design Reference	Code File Reference	Test Case ID	System Test Case ID
1	REQ-1.1	Select network interface	Capture Layer	TBD	TBD	TBD	TBD
2	REQ-1.2	Start/stop capture session	Capture Layer	TBD	TBD	TBD	TBD
3	REQ-1.6	Write raw .pcap to storage	Capture Layer / Storage Layer	TBD	TBD	TBD	TBD
4	REQ-2.1	Read Zeek structured logs	Zeek Integration Layer	TBD	TBD	TBD	TBD
5	REQ-2.2	Protocol breakdown display	Python UI Layer	TBD	TBD	TBD	TBD
6	REQ-3.2	Use Zeek detection scripts	Zeek Integration Layer	TBD	TBD	TBD	TBD
7	REQ-3.3	Generate alert entries	Python UI Layer	TBD	TBD	TBD	TBD
8	REQ-4.1	Maintain local session index	Storage Layer	TBD	TBD	TBD	TBD
9	REQ-4.3	Export session data	Storage Layer	TBD	TBD	TBD	TBD